CODING

Effective unit-testing in Python – with examples

How to implement effective unit-testing in Python to ensure production-level, bug-free code.

Christopher Karg
Oct 1, 2023 • 18 min read

Source: https://www.pexels.com/photo/red-vintage-car-stopped-on-the-crossroads-15706251/

My next article is on the subject of unit-testing in Python. It is a topic that I found is often overlooked in lessons, books and online tutorials. It is however a crucial skill when creating production-level, bullet-proof code. The scenarios in this article are somewhat skewed toward the world of data/data science. As such, my opinion of an 'effective' unit-test may differ to someone coming from a different background.

> The ultimate aim of writing unit-tests is to prevent bugs being pushed into production.

The result of which leads to many headaches and time to resolve. Different companies have different testing methodologies. Some projects I have worked on (specifically web-scraping projects) have not required unit-testing per se but can benefit from implementing python's inbuilt logging functionality and try/except handling. This article only covers the practical implementations of unit-testing along with examples and links to code that are available in my GitHub repo. Feel free to clone it and have a play with some of the testing functions – instructions on how to do so can be found in the README of the repo. The repo is split into Easy, Medium and Hard categories, each building upon the concepts covered in the previous category.

The easy and medium examples in this article go over testing syntax and how to use certain features of the unit-testing library. The harder example builds on the easy and medium examples but also introduces the idea of '***effective***' unit-testing and what exactly you should be testing. It is all well and good if you are writing unit-tests but if you are testing for irrelevant things, the unit-tests themselves become somewhat irrelevant!

In a production environment, these tests would be incorporated into a CI/CD pipeline so they run automatically when code is updated. This is somewhat outside the scope of the article but in summary the general process would be as follows:

· Create test scripts · Create a configuration file for your chosen CI/CD service (GitHub Actions, Travis CI, Jenkins etc.) The config file allows you to specify a task name, what commands are to be run within the task and the location of the scripts you want to run · Push config file and test scripts to code repo (GitHub/GitLab) · If the CI/CD service is correctly set-up the tests will be incorporated into the pipeline. · Notifications of failed tests can be sent out via Slack or Email to ensure any issues can be resolved quickly.

I will embed the relevant code for each category at the start of each section. A link to the respective GitHub repo will also be included.

We'll start with the Easy category.

Easy

- General unit-testing.
- How to set-up and run testing functions.

In the repo I have created two functions. One that simply adds two numbers. The 2nd function computes the factorial of a given number:

```
def add_numbers(a, b):
    return a + b

def factorial(n):
    """
    Function to perform factorial operations. eg:
    0! = 1 (by convention, the factorial of 0 is defined to be 1).
    1! = 1 (since there is only one positive integer from 1 to 1).
    2! = 2 x 1 = 2.
    3! = 3 x 2 x 1 = 6.
    4! = 4 x 3 x 2 x 1 = 24.
    """
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbe
    if n == 0:
        return 1
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Let's swap our attention to the testing script (tests.py):

```
import unittest
from math_functions import add_numbers, factorial

class TestFunctions(unittest.TestCase):
    #Below functions test the add_numbers function
    def test_addition_positive_numbers(self):
        self.assertEqual(add_numbers(2, 3), 5)

    def test_addition_negative_numbers(self):
        self.assertEqual(add_numbers(-2, -3), -5)

    #Below functions test the factorial function
    def test_factorial_zero(self):
        self.assertEqual(factorial(0), 1)

    def test_factorial_positive_number(self):
        self.assertEqual(factorial(5), 120)

    def test_factorial_negative_number(self):
        with self.assertRaises(ValueError):
            factorial(-2)

if __name__ == '__main__':
    unittest.main()
```

Notice I have imported the functions I want to test at the top of the tests.py script along with python's unittest library. Within the this script you can see 1 class has been instantiated, along with 5 different functions. Each function serves as a single test. Therefore in this example I am writing 5 unit-tests for a script containing 2 functions.

Again depending on your desired testing methodology, you may choose to write one meaningful test per function, multiple tests per function (encouraged if the function is complex), or to combine it with log messages and try/except blocks (as you will see in the latter examples) to catch all cases where your code may break. In the interests of risk aversion, I would advise writing as many meaningful/effective tests as possible. The idea of what constitutes an effective test is discussed later in the article.

As I am testing fairly straightforward functions, their logic is not that complex. We simply perform specific computations on the function inputs. The most effective way to test these functions is by checking the output is what we expect. Take the first example – we expect the output of 2+3 to be 5. To write check using the unittesting library, we use the *self.assertEqual* method. Another term for this type of test is an 'assertion test'.

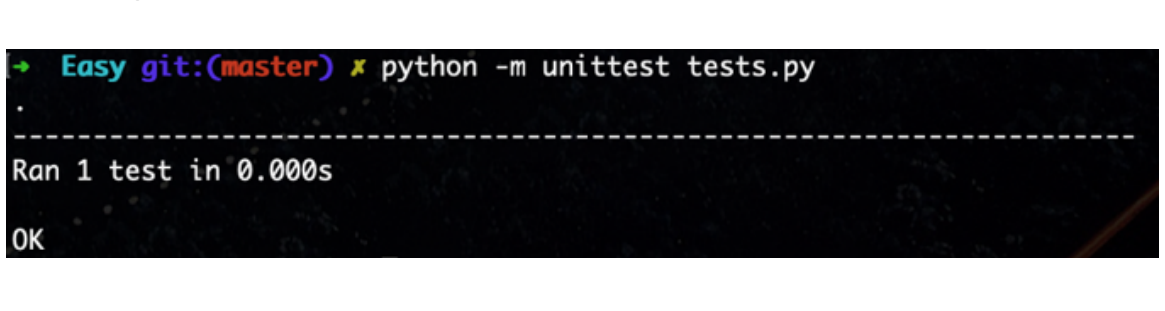
We have our first unit-test written. Let's check it works.

From within the Easy directory run:

python -m unittest tests.py

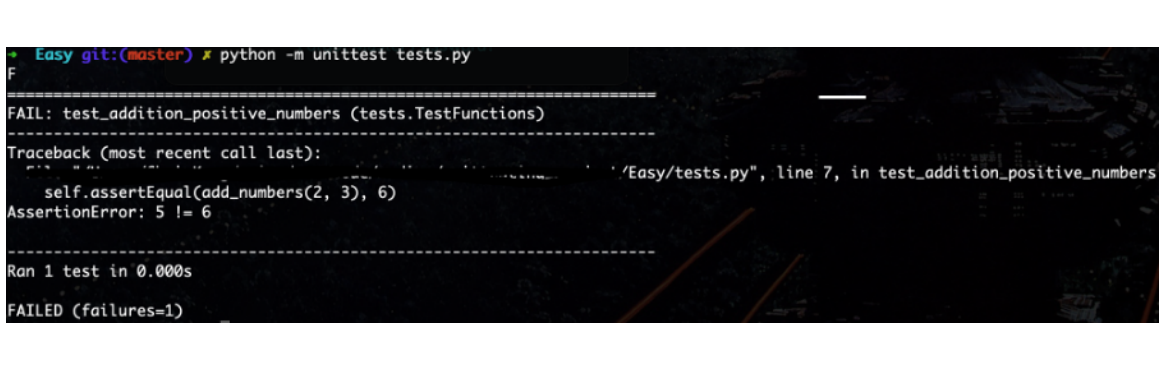
The above command is us telling the python interpreter to run the tests.py script using the unittest module. The ***-m*** stands for module.

The output of our command looks as follows:



```
→  Easy git:(master) ✗ python -m unittest tests.py
.
----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
```

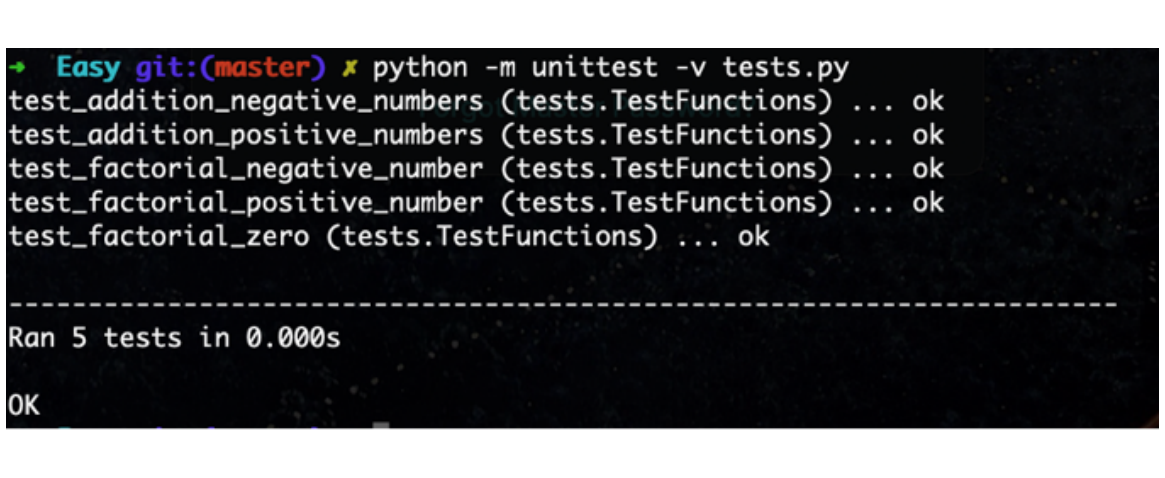
For the sake of testing let's see the output if the unit-test were to fail. I will change the assertion so that 2+3=6 (incorrect…):



```
→  Easy git:(master) ✗ python -m unittest tests.py
F
======================================================================
FAIL: test_addition_positive_numbers (tests.TestFunctions)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "                                 /Easy/tests.py", line 7, in test_addition_positive_numbers
    self.assertEqual(add_numbers(2, 3), 6)
AssertionError: 5 != 6

----------------------------------------------------------------------
Ran 1 test in 0.000s

FAILED (failures=1)
```

You can see the above test failed. The output of the function was 5. We told the system we were expecting 6. As you can see the output is very verbose by default and allows you to fairly quickly get to the bottom of the issue.

Let's run all of our 5 tests for the 2 functions and check the output. This time we will add the ***-v*** marker to increase verbosity:



```
→  Easy git:(master) ✗ python -m unittest -v tests.py
test_addition_negative_numbers (tests.TestFunctions) ... ok
test_addition_positive_numbers (tests.TestFunctions) ... ok
test_factorial_negative_number (tests.TestFunctions) ... ok
test_factorial_positive_number (tests.TestFunctions) ... ok
test_factorial_zero (tests.TestFunctions) ... ok

----------------------------------------------------------------------
Ran 5 tests in 0.000s

OK
```

You can see that adding the ***-v*** marker makes it easy to see exactly which testing functions are being called. It's always a good feeling when all tests pass ▯

One testing function I'd like to point out is the following:

```
def test_factorial_negative_number(self):
        with self.assertRaises(ValueError):
            factorial(-2)
```

If you check back to our original factorial function:

```
def factorial(n):
    """
    Function to perform factorial operations. eg:
    0! = 1 (by convention, the factorial of 0 is defined to be 1).
    1! = 1 (since there is only one positive integer from 1 to 1).
    2! = 2 x 1 = 2.
    3! = 3 x 2 x 1 = 6.
    4! = 4 x 3 x 2 x 1 = 24.
    """
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbe
    if n == 0:
        return 1
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

You can see that a factorial computation is not possible for any number less than 0. As per the above code we wanted to raise a ValueError with a comment if this error is triggered. If we are taking the effort to catch errors in our code, we must also make the effort to trigger these errors in our tests. We ultimately need to ensure our function also fails as intended. The test of this function is also to be considered an assertion test, however rather than asserting output == expected value, we are testing to ensure the ValueError is raised as intended. There are other types of assertion test all listed within python's extensive unit-testing documentation.

Let's move to the more challenging tests.

Medium

- Patching function outputs

Move into the Medium directory for these tests. Here we build upon the previous examples of assertion tests and begin to patch the function outputs we have created within our **_get_weather_data.py** *script. This script looks to replicate a typical request to an imaginary weather API and returns a response JSON that we will look to analyse within our script. The* **get_weather_data.py_** is as follows:

```
import requests

def get_weather_data(city):
    """
    Simulates an API call to fetch weather data
    """
    response = requests.get(f'https://api.weather.com/data/{city}')
    if response.status_code == 200:
        return response.json()
    else:
        raise Exception("Failed to fetch weather data")

def analyze_weather(city):
    """
    Perform analysis on weather data
    """
    data = get_weather_data(city)

    if data['temperature'] > 25 and data['humidity'] < 70:
        return "Hot and dry"
    elif data['temperature'] < 10:
        return "Cold"
    else:
        return "Moderate"
```

The use case for this may not be immediately obvious but it is very useful for testing your code logic if you do not actually have access to the API in question. Since we are making a request to an imaginary API, the function will not return any data. How are we supposed to test the remaining code if it breaks after the first line? This is where patching comes in handy.

Let's move our attention to the testing script to see how it works:

```
import unittest
from unittest.mock import patch
from get_weather_data import analyze_weather

class TestGetWeatherData(unittest.TestCase):
    @patch('get_weather_data.requests.get')
    def test_analyze_weather_hot_dry(self, mock_get):
        mock_response = mock_get.return_value
        mock_response.status_code = 200
        mock_response.json.return_value = {
            'temperature': 30,
            'humidity': 60
        }
        result = analyze_weather('city')
        self.assertEqual(result, "Hot and dry")

    @patch('get_weather_data.requests.get')
    def test_analyze_weather_cold(self, mock_get):
        mock_response = mock_get.return_value
        mock_response.status_code = 200
        mock_response.json.return_value = {
            'temperature': 5,
            'humidity': 80
        }
        result = analyze_weather('city')
        self.assertEqual(result, "Cold")

    @patch('get_weather_data.requests.get')
    def test_analyze_weather_moderate(self, mock_get):
        mock_response = mock_get.return_value
        mock_response.status_code = 200
        mock_response.json.return_value = {
            'temperature': 20,
            'humidity': 50
        }
        result = analyze_weather('city')
        self.assertEqual(result, "Moderate")

    @patch('get_weather_data.requests.get')
    def test_analyze_weather_api_failure(self, mock_get):
        mock_get.return_value.status_code = 404
        with self.assertRaises(Exception):
            analyze_weather('city')

if __name__ == '__main__':
    unittest.main()
```

Check the imports. They're similar to the easy example, only now we have also included the patch decorator that is imported from the ***unittest.mock*** library. Let's take the first test as an example and go through the syntax:

```
class TestGetWeatherData(unittest.TestCase):
    @patch('get_weather_data.requests.get')
    def test_analyze_weather_hot_dry(self, mock_get):
        mock_response = mock_get.return_value
        mock_response.status_code = 200
        mock_response.json.return_value = {
            'temperature': 30,
            'humidity': 60
        }
        result = analyze_weather('city')
        self.assertEqual(result, "Hot and dry")
```

As per the previous example, we create out test class. Then we implement a function decorator. Decorators modify/extend the behaviour of a function – I'd suggest researching a little if you are new to the concept. In this example, we use the decorator to 'mock' the behaviour of the requests.get function call in our **_get_weather_data.py** *script. This allows us to control the behaviour of the function call and change the return value. This way we can insert an expected return value/s without actually making a call to the imaginary API (which would not work anyway). This allows us to test our function logic without needing access to the API. The calls to the function are replaced with a mock object –* **mock_get_**.

The next line looks like a regular function definition, the only thing here is that we are passing the **_mock_get** *parameter (the object that is being used to replace the* **requests.get_** function call). This allows us to inject the mock object into the function for later use.

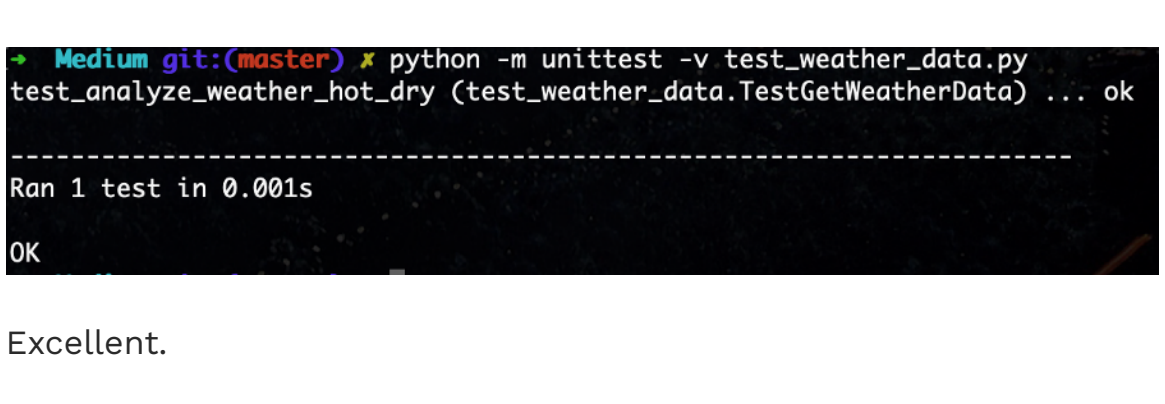
Next we create a **_mock_response_** object as the return value of the mock*get object. Any call to* **mock_get_** *will return this* **mock_response_** object.

Now we will set certain variables within our defined function to an expected output. This will allow us to test the logic of the function without actually making a call to the API. We assign mock_response.status*code = 200. This simulates a successful HTTP response as defined in our* **get_weather_data()** *function within our* **get_weather_data.py_** script. We also assign a JSON response to mock_response.json.return_value. This again simulates the behaviour of the function we want to test.

Next we make an actual call to the main function within the script. Notice this function calls the function for which we previously mocked the return values. We are now able to run this function with the expected (fake) return values to test the overall logic. The output is saved to the 'result' variable.

So take a moment to double check what variables we mocked above and how this should affect the output of the function. The **_status_code = 200**, *the JSON returns a value of* **{'temperature': 30, 'humidity': 60}**. *When checking the logic of the actual function we are testing* (**analyze_weather()_**), we want the return value to be "Hot and dry", thus we create an assertion test, passing in the result variable and checking it equals the expected function output.

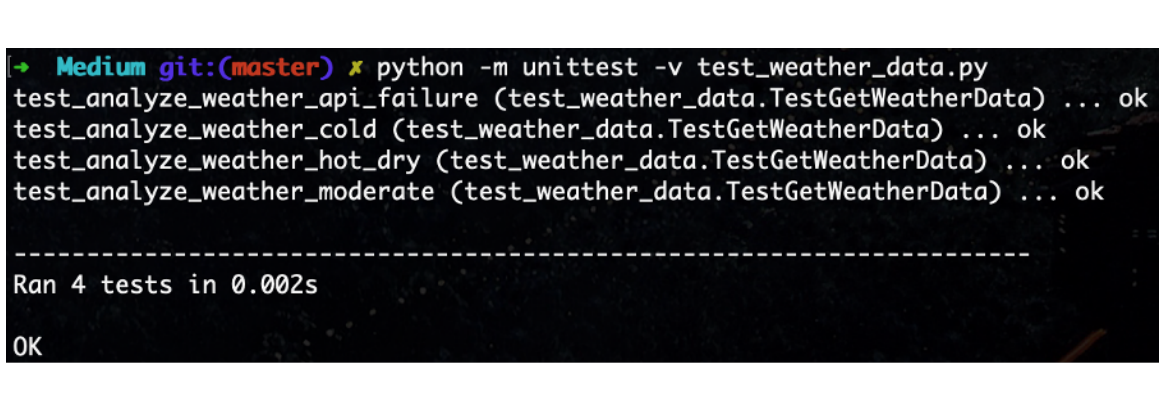
When running this test with the verbosity marker we get the following output:



```
→  Medium git:(master) ✗ python -m unittest -v test_weather_data.py
test_analyze_weather_hot_dry (test_weather_data.TestGetWeatherData) ... ok

----------------------------------------------------------------------
Ran 1 test in 0.001s

OK
```

Excellent.

The 3 further testing functions use a similar methodology to the easy problem, we are checking how various inputs trigger the expected logic within the functions through assertion checks. In the final test we are triggering an error by passing in a 404 status code to mimic a failed API call. We want this to trigger our Exception clause within the **_get_weather_data()_** function.



```
→  Medium git:(master) ✗ python -m unittest -v test_weather_data.py
test_analyze_weather_api_failure (test_weather_data.TestGetWeatherData) ... ok
test_analyze_weather_cold (test_weather_data.TestGetWeatherData) ... ok
test_analyze_weather_hot_dry (test_weather_data.TestGetWeatherData) ... ok
test_analyze_weather_moderate (test_weather_data.TestGetWeatherData) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.002s

OK
```

Which we succeed in doing.

Let's move to the hard example.

Hard

- 'Effective' unit-testing
- Side-effect
- Using a sample JSON file/data as the return value of an API call.

The repo for the Hard example can be found here. We will cover the examples after a few words on ***effective*** unit-testing.

What makes an ***effective*** unit-test? As with many things in data science the answer depends on your scenario. In my opinion unit-tests come into their own in the initial stages of a project i.e. when you are gathering data or writing ETL scripts to handle data transfer from a 3rd party or an internal database. This allows you to track the quality of your data and ensure it is formatted the way you are expecting. Of course further downstream transformations probably need to occur and more unit-tests need to be implemented elsewhere in your pipeline but it helps to know you are working with the correct data in the first place. I believe effective unit-tests in this initial stage should cover the following areas:

· **Is the schema correct?** Are the correct number of columns present? Are all required columns present? Does the data actually contains records? Changes to the schema can lead to data integration issues/bugs

· **The values within the data** Check for a certain number of null values A lot of discrepancies/anomalies can indicate data quality issues and can be raised to the owner of the data source.

· **Semantics** Do columns have the expected dtypes or units? Is your ***distance*** column in miles as expected or has it been changed to km? Is the **_employee_count_** column an integer as expected or is it a float? Your calculations would be thrown off if something like this were to be misinterpreted.

The idea of testing for these specific criteria can be linked to the idea of data contracts, which include the above pointers as well as monitoring SLA's (focussing on data accessibility/availability). If you are interested in the topic I'd advise looking through some of Chad Sanderson's content on LinkedIn. All of the above fits into the bigger picture of quality assurance and regulatory requirements (GDPR etc.)

Another tip is to ensure that your code fails gracefully and actually produces the descriptive error messages you or another developer require to fix the error. By writing tests to ensure graceful failure you will save yourself and others a lot of time.

Coming back to the code…

Check the code within the **_get_company_data.py_** script:

```
import requests

def get_company_data(company_name):
    try:
        response = requests.get(f'https://api.example.com/companies/{
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        raise Exception(f"Failed to fetch company data: {e}")

def analyze_company(company_name):
    try:
        data = get_company_data(company_name)
        if not data:
            raise Exception("Company 'non_existent_company' not found

        # Check if the confidence score is 0.9 or higher
        confidence_score = data.get("confidence_score", 0)  # Default
        if confidence_score < 0.9:
            raise Exception("Company does not meet the confidence sco

        # Check schema
        required_fields = ["name", "revenue", "employees", "industry"
        for field in required_fields:
            if field not in data:
                raise Exception(f"Missing '{field}' in company data")

        # Perform further analysis on data below.......
        #
        #
        #

        return f"Analysis result for {data['name']}"

    except Exception as e:
        raise Exception(f"Failed to analyze company data: {e}")
```

You can see it is very similar to our previous example only here we use slightly different logic where we are also checking for schema related issues. I have left space in the script for you to edit and possibly implement some of the checks I listed in the bullet points above.

A big difference here is that we import a JSON file containing what could be a sample output from an API. Check the JSON file within this directory to see how it is formatted:

```
 {
  "company_1": {
      "name": "ABC Inc.",
      "revenue": 1000000,
      "employees": 50,
      "industry": "Technology",
      "location": "New York",
      "confidence_score": 0.8,
      "leadership": {
          "ceo": "John Doe",
          "cto": "Jane Smith"
      },
      "products": [
          {
              "name": "Product A",
              "category": "Software"
          },
          {
              "name": "Product B",
              "category": "Hardware"
          }
      ]
  },
  "company_2": {
      "name": "XYZ Ltd.",
      "revenue": 500000,
      "employees": 20,
      "industry": "Finance",
      "location": "London",
      "confidence_score": 0.9,
      "leadership": {
          "ceo": "Alice Johnson",
          "cfo": "Bob Williams"
      },
      "products": [
          {
              "name": "Product X",
              "category": "Finance Software"
          }
      ]
  }
}
```

This method is how one would potentially test their code using a sample output from an API endpoint provided by a 3rd party provider. Simply import the JSON and mock the output of the ***requests.get*** function which you can see we have done in lines 10+11 of the test script.

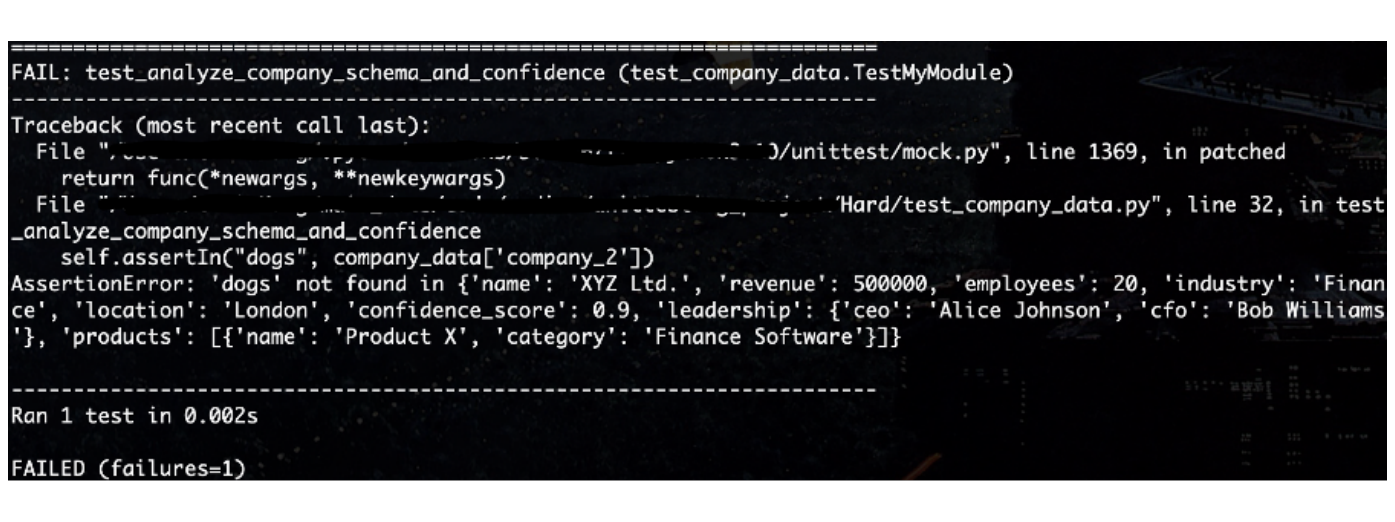
If you look through the top of the test script it contains very similar syntax to what we covered in our previous examples:

```
import unittest
import json
from unittest.mock import patch, Mock
from get_company_data import analyze_company

class TestMyModule(unittest.TestCase):
    @patch('get_company_data.requests.get')
    def test_analyze_company_schema_and_confidence(self, mock_get):
        # Load data from the fake_company_data.json file
        with open('fake_company_data.json', 'r') as file:
            company_data = json.load(file)

        # Mock the response for an existing company with confidence s
        mock_response = Mock()
        mock_response.status_code = 200
        mock_response.json.return_value = company_data['company_2']
        mock_get.return_value = mock_response

        # Test for an existing company with a confidence score of 0.9
        result = analyze_company('company_2')
        self.assertEqual(result, "Analysis result for XYZ Ltd.")

        # Check schema keys for company_2
        self.assertIn("name", company_data['company_2'])
        self.assertIn("revenue", company_data['company_2'])
        self.assertIn("employees", company_data['company_2'])
        self.assertIn("industry", company_data['company_2'])
        self.assertIn("location", company_data['company_2'])
        self.assertIn("confidence_score", company_data['company_2'])
        self.assertIn("leadership", company_data['company_2'])
        self.assertIn("products", company_data['company_2'])
        #uncomment below test to see how test fails
        #self.assertIn("dogs", company_data['company_2'])

        # Check confidence score for company_2
        confidence_score = company_data['company_2']["confidence_scor
        self.assertTrue(0.9 <= confidence_score <= 1, "Confidence sco

        # Mock the response for a non-existent company
        mock_response = Mock()
        mock_response.status_code = 404
        mock_response.json.side_effect = Exception("JSON decoding fai
        mock_get.return_value = mock_response

        # Mock the response for an existing company with confidence s
        mock_response = Mock()
        mock_response.status_code = 200
        mock_response.json.return_value = company_data['company_1']
        mock_get.return_value = mock_response

        # Test for an existing company with confidence score 0.8
        with self.assertRaises(Exception) as context:
            analyze_company('company_1')
        self.assertIn("Company does not meet the confidence score thr

if __name__ == '__main__':
    unittest.main()
```

For the sake of experimentation, I have adapted the syntax somewhat from line 14 onwards. Rather than creating a **_mock_response object** *by setting it equal the* **return_value** *of the mock object (***mock_get***), I have simply create a response object by calling the imported* **Mock_** class directly. I would consider creating the mock_response object using mock_get.return_value to be the better practice and more pythonic – after all explicit is better than implicit.

Once the mock object is created, we assign the return values and assign the data for company2 *to be the initial return value as we want to double check the schema provided is correct. Again we also modify the behaviour of the* **requests.get_** function call to return the mock response.

Once we call the function in line 20 we start our assertion checks. Here we are ultimately checking the data contains the columns we might require as input to downstream algorithms. It is at this stage that if the data does not contain a desired datapoint, the tests would fail as we see below (for the sake of testing I have included a check for a column named '***dogs***'):



```
======================================================================
FAIL: test_analyze_company_schema_and_confidence (test_company_data.TestMyModule)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "[redacted]/unittest/mock.py", line 1369, in patched
    return func(*newargs, **newkeywargs)
  File "[redacted]Hard/test_company_data.py", line 32, in test
_analyze_company_schema_and_confidence
    self.assertIn("dogs", company_data['company_2'])
AssertionError: 'dogs' not found in {'name': 'XYZ Ltd.', 'revenue': 500000, 'employees': 20, 'industry': 'Finan
ce', 'location': 'London', 'confidence_score': 0.9, 'leadership': {'ceo': 'Alice Johnson', 'cfo': 'Bob Williams
'}, 'products': [{'name': 'Product X', 'category': 'Finance Software'}]}

----------------------------------------------------------------------
Ran 1 test in 0.002s

FAILED (failures=1)
```

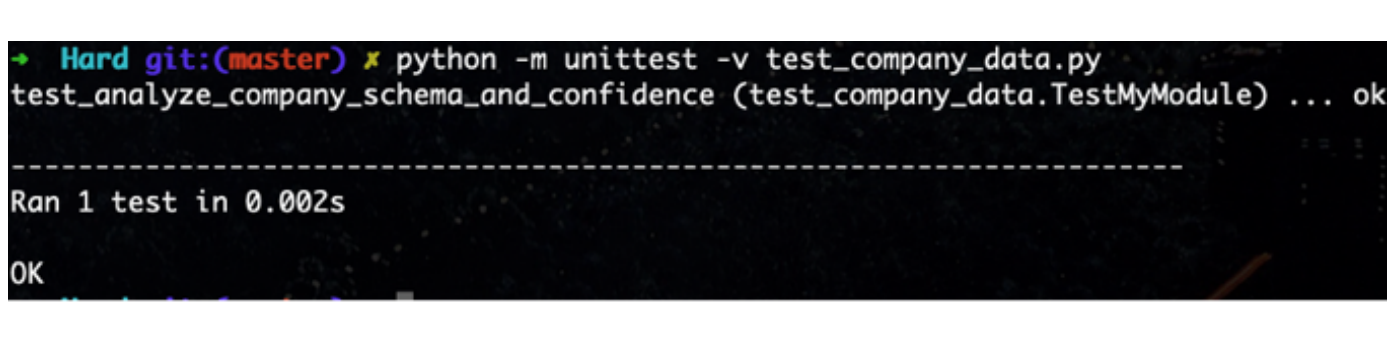
As we can see the assertion test failed as '***dogs***' was not found in the schema.

On line 40 we see a new mock_response is created as we want to test the result of a 404 error. It is in this code that I introduce the concept of side_effect (line 42). In Python, the side_effect attribute of a mock object is used to define custom behaviour of a specific method or function. It allows you to define what should happen if the method is invoked such as raising exceptions or returning specific values etc. A common use is to test different behaviours for multiple calls to the same method. I'll include a brief example for ease of understanding below:

```
from unittest.mock import Mock

api_client = Mock()

# Define different behaviors for successive calls to the 'get_data' me
api_client.get_data.side_effect = [10, 20, 30]

# Call 'get_data' method three times
result1 = api_client.get_data()  # Returns 10
result2 = api_client.get_data()  # Returns 20
result3 = api_client.get_data()  # Returns 30

print(result1, result2, result3)  # Output: 10 20 30
```

Let's return to our original company example in **_test_company_data.py**. *In line 42 we set the* **side_effect** *object to an exception with a custom message. That means when we later call* **mock_repsonse.json()***, it will raise this custom exception. Finally we set the* **mock_get.return_value_** equal to the previously coded return values of the mock_response object.

Finally, from line 52 onwards, we set-up a context manager using a self.assertRaises method. It specifies that an exception of type 'Exception' is expected to be raised within this code block. This makes sense as company_1 (the company we are testing within this new mock_object) has a confidence_score of 0.8. Our code should only accept companies with a confidence score of 0.9 or higher, otherwise an Exception is thrown. This is the desired behaviour and we are checking to confirm this is indeed the case. We check to see if the string output from our Exception contains the specified message.



```
→  Hard git:(master) ✗ python -m unittest -v test_company_data.py
test_analyze_company_schema_and_confidence (test_company_data.TestMyModule) ... ok

----------------------------------------------------------------------
Ran 1 test in 0.002s

OK
```

Great!

Thanks for sticking around. When writing your unit-tests, remember to consider not only the logic of your code but also the bigger picture – how do the scripts you are testing fit into your current project. This will help you to formulate more ***effective*** unit-tests.

Let me know if you have any questions or wish to discuss any of the above.

All images by author unless otherwise noted.

* * *

WRITTEN BY

Christopher Karg

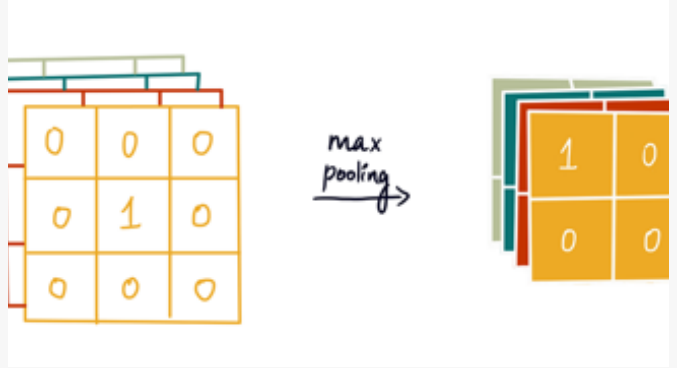
See all from Christopher Karg

Topics:

Coding Data Science Python Software Development

Share this article: ○ ○ ○

Related Articles



ARTIFICIAL INTELLIGENCE

Implementing Convolutional Neural Networks in TensorFlow

Step-by-step code guide to building a Convolutional Neural Network

Shreya Rao
August 20, 2024 • 6 min read

DATA SCIENCE

Hands-on Time Series Anomaly Detection using Autoencoders, with Python

Here's how to use Autoencoders to detect signals with anomalies in a few lines of…

Piero Paialunga
August 21, 2024 • 12 min read

DATA SCIENCE

Solving a Constrained Project Scheduling Problem with Quantum Annealing

Solving the resource constrained project scheduling problem (RCPSP) with D-Wave's hybrid constrained quadratic model (CQM)

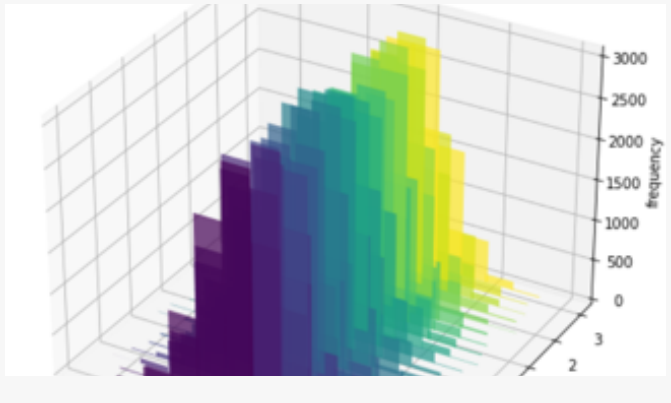
Luis Fernando PÉREZ ARMAS, Ph.D.
August 20, 2024 • 28 min read

DATA SCIENCE

Back To Basics, Part Uno: Linear Regression and Cost Function

An illustrated guide on essential machine learning concepts

Shreya Rao
February 3, 2023 • 6 min read



DATA SCIENCE

Must-Know in Statistics: The Bivariate Normal Projection Explained

Derivation and practical examples of this powerful concept

Luigi Battistoni
August 14, 2024 • 7 min read

DATA SCIENCE

How to Make the Most of Your Experience as a TDS Author

A quick guide to our resources and FAQ

TDS Editors
September 13, 2022 • 4 min read

DATA SCIENCE

Our Columns

Columns on TDS are carefully curated collections of posts on a particular idea or category…

TDS Editors
November 14, 2020 • 4 min read

DATA SCIENCE

Optimizing Marketing Campaigns with Budgeted Multi-Armed Bandits

With demos, our new solution, and a video

Vadim Arzamasov
August 16, 2024 • 10 min read

DATA SCIENCE

Back to Basics, Part Tres: Logistic Regression

An illustrated guide to everything you need to know about Logistic Regression

Shreya Rao
March 2, 2023 • 8 min read

Your home for data science and AI. The world's leading publication for data science, data analytics, data engineering, machine learning, and artificial intelligence professionals.

© Insight Media Group, LLC 2025

ABOUT · PRIVACY POLICY · TERMS OF USE

Sign up to our newsletter

Your email*

First name*

Last name*

Job title*

Job level*

Please Select

Company name*

☐ I consent to receive newsletters and other communications from Towards Data Science publications.*

Subscribe Now